撮って送るだけ帳簿
発表台本（7分）

丹内 智弥　2026年10月　|　本編スライド11枚　|　発表7分（セリフ約2,000字）+ 質疑3分

1. Laravelを使うポイント

この章は読み上げない。スライド7（3:28〜4:42）で話す内容の下準備と、質疑で聞かれたときの答えに使う。

発表で言う一文：Laravelは「作るための道具」ではなく、「間違いを確定させないための道具」として使う。

経理アプリで一番怖いのは、間違った数字が、誰も気づかないうちに確定することだ。Laravelは、これを止める仕組みを標準機能として持っている。だから、少人数の開発でも守りを固められる。発表では次の4つを、「機能名 → 何をするか → 何が防げるか」の順で言う。

① Fortify（ログイン）

メール確認、2段階認証、パスワードの復旧を、自作せずに組み込める。画面とのやり取りはSanctum のCookie認証を使う。防げるのは、なりすましのログイン。

② Policy（権限）

「仕訳を作るのは経理担当、承認できるのは社長だけ」「自分で作った仕訳は自分で承認できない」を、サーバー側で判定する。画面のボタンを隠すだけでは、URLを直接呼べば通ってしまう。Policy なら、どの経路から来ても同じ判定がかかる。防げるのは、権限のない人の承認と自己承認。

③ DB::transaction と行ロック（確定）

承認・取消・月次の締めは、DB::transaction の中で lockForUpdate を使って実行する。承認ボタンを2回押しても、2人が同時に押しても、仕訳は1回しか確定しない。途中で失敗すれば全体が元に戻り、片側だけ保存されることもない。防げるのは、二重の確定と、半端な保存。

④ Pest（テスト）

「他の会社のIDに書き換えても見えない」「同じ送信を2回しても1件」「締めた月は変更できない」を自動テストにし、変更のたびに確かめる。防げるのは、直したつもりで壊すこと。

発表では話さないが、質問されたら答える補足

全テーブルに会社IDの条件を自動で付ける Eloquent のグローバルスコープ、金額は整数円・日付は暦日だけを受け付ける Form Request、通帳の明細の取り込みやバックアップを裏で実行する Queue（同じ明細は一意キーでスキップする）。いずれも同じ考え方で、間違いを入口で止めるために使う。

2. 台本本文

表記：〔 〕は話を止める場所と秒数。「演出」は動きのメモで、声に出さない。

スライド1　表紙（0:00〜0:26）

レシートは、撮って送るだけ。〔2秒〕

今日は、この一言だけ持って帰ってください。

家族で経営する小さな会社のための経理アプリ、「撮って送るだけ帳簿」です。最初に使うのは、岩手で産業用機械の修理・溶接・加工をしている、丹内建機サービスです。

演出：話し始める前に会場を見て一拍置く。最初の一言は、ゆっくり大きく。

スライド2　想定ユーザーと課題（0:26〜1:09）

皆さんに、ひとつ質問です。

いま、財布の中で眠っているレシートは、何枚ありますか？〔3秒〕

月末、こんな会話が聞こえてきます。

「ねえ、あのガソリン代のレシートある？」「あ……作業着のポケットだ」〔1秒〕

レシートは、1か月ポケットで熟成されます。〔1秒・笑いを待つ〕

それを経理担当が、1枚ずつ紙の帳簿へ書き写す。家族に何度も「レシート出して」と言うのも、地味にしんどい仕事です。

まだ仮説なので、12月までに聞いて確かめます。

演出：会話は2人の声を少し変えて言う。「熟成」のあとは笑いを待ち、急がない。

スライド3　今の代替手段（1:09〜1:42）

では、今の方法ではだめなのか。

紙、Excel、会計ソフト、税理士。どれにも良いところがあります。

でも、共通点がひとつあります。レシートを受け取ったあとの作業が、全部、経理の1人に集まることです。

仮に月20時間なら、1年で240時間。まる10日分です。〔1秒〕

足りないのは、計算ではありません。現場の人が、その場で渡せる「入口」です。

演出：既存の方法を否定しない。「まる10日分」は、ゆっくり言う。

スライド4　コンセプト（1:42〜2:35）

そこで、このアプリの考え方です。

会計ソフトは、「どう速く入力するか」で競っています。私は、問いを変えました。そもそも、入力しなくていいデータはないか。レシートを一番早く持っているのは、誰か。〔1秒〕

答えは、通帳の明細と、現場の人です。

だから、torudakeは家族で回す道具です。名前のtorudakeは「撮るだけ」。現場の人への約束です。

現場の人は、撮って送るだけ。ボタン1つです。

経理担当は、AIの下書きを、通帳の明細と突き合わせて直すだけ。今の帳簿の欄は、そのまま使えます。

社長は、確かめて締めるだけ。確認の目が2つ入ります。

演出：2つの問いは、会場に投げかけるように言い、1秒待ってから答える。

スライド5　主な機能（2:35〜3:13）

機能は3つに絞りました。集める、直す、締める。

1つ目、集める。レシートは撮って送る。通帳は明細データを取り込む。現金は、月末に数えて入れる。

2つ目、直す。AIが写真から下書きを作り、明細と突き合わせます。人が直した理由は、次からAIが使います。承認されるまで、帳簿には入りません。

3つ目、締める。未処理、下書き、通帳と現金の差。全部が0になったら、その月を締めます。

スライド6　イメージ図（3:13〜3:28）

画面は、この3枚です。左が現場の画面。ボタンは1つだけです。真ん中が経理の画面。右が、月の初めに社長が見る画面で、大きく「あと3件」と出ます。

演出：左・真ん中・右の順に手で指す。説明より見せる時間を長くとる。

スライド7　なぜLaravelか（3:28〜4:42）

経理アプリで一番怖いのは、間違った数字が、誰も気づかないうちに「確定」してしまうことです。〔1秒〕

だから、裏側にLaravelを選びました。止める仕組みが、標準で揃っているからです。

1つ目、Fortify。ログインです。2段階認証を、自作せずに入れられます。

2つ目、Policy。権限です。「作るのは経理担当、承認するのは社長だけ」。これを画面ではなく、サーバー側で判定します。ボタンを隠すだけでは、URLを直接呼べば通ってしまうからです。

3つ目、DB::transaction と行ロック。確定です。承認ボタンを2回押しても、2人が同時に押しても、仕訳は1回しか確定しません。

4つ目、Pest。他社のデータが見えないことを、毎回自動で確かめます。

つまり、Laravelは「作るための道具」ではなく、「間違いを確定させないための道具」として使います。

演出：各項目は「機能名 → 何をするか」の順で区切って言う。時間が押したら4つ目を「テストも標準で書けます」の一言にする。

スライド8　使う場面（4:42〜5:25）

では、1か月の流れを想像してみてください。〔2秒〕

現場の帰り、社長が給油して、その場でパシャっと撮って送る。10秒です。

週に1回、経理担当が通帳の明細を取り込み、AIの下書きを直す。そのあと社長が確かめて、承認する。

月末に現金を数えて、翌月の初め、画面には「あと0件」。確かめて、締める。

もう、月末に「レシート出して」と言わなくていい。レシートは、もうポケットの中ではありません。その場で送られています。

演出：ここだけ少し語りかけるように。最後の一文で冒頭の問いを回収する。

スライド9　なぜこのアイデアか（5:25〜6:00）

なぜ、このアイデアにしたのか。

私はこれまで、家業で見積もり、給与計算、出勤管理の仕組みを作ってきました。動くものは、作れました。

でも、作る中で気づきました。「作れる」ことと、「使ってもらえる」ことは、違う。〔1秒〕使う人の困りごとを先に聞かないと、本当に必要なものにはならない。

だから今回は、簿記を勉強しながら、まず経理の流れを聞くところから始めます。

演出：「違う」の前後で間をとり、少しゆっくり話す。

スライド10　成功の基準（6:00〜6:30）

成功は、減った時間で判断します。

最初の一歩は、アプリを作らないことです。〔1秒〕2週間、LINEでレシートを送るだけで試し、写真が届く割合を測ります。

目標は、2027年9月までに30%減。月20時間なら14時間です。1年で72時間、まる3日分の時間が、本業に戻ります。この数字は、測ってから見直します。

演出：「アプリを作らない」で一拍置く。「まる3日分」は、3枚目の「まる10日分」と対にして明るく言う。

スライド11　まとめの一言（6:30〜7:00）

皆さんの身近な会社にも、ポケットで熟成中のレシートがあるかもしれません。もし心当たりがあれば、紹介してください。話を聞きに行きます。

このアプリで、家族で経営する会社の「月末にまとめて書き写す夜」を、なくせたら嬉しいです。〔2秒〕

レシートは、撮って送るだけ。

ありがとうございました。

演出：お願いは小さく、具体的に。最後の一言のあと2秒止まってから礼をする。

3. 質疑応答（3分）

答え方の型：①質問を一度くり返す → ②弱点を先に認める → ③「だから、こうする」で対処を言う → ④当面どうするか、いつ確かめるかで終える。1問30秒以内。最後のお願いは小さく（「2週間、撮って送ってみてください」）。

聞き手は会計やアプリに詳しい人たち。知識のある相手には、弱点を隠すより先に言うほうが信用される（両面提示）。分からないことは推測で答えず、「確かめてからお伝えします」と言う。

Q1. AIの読み取りは、精度が安定しないのでは？（付録16）

はい、100%にはなりません。感熱紙の色あせやしわ、手書きは特に間違えますし、AIは自信がありそうに間違えることもあります。どれくらい読めるかも、まだ測っていません。

だから、間違えても困らない作りにしています。通帳とカードは写真を読まず、明細データで取り込みます。AIの候補は下書きまでで、明細や現金と突き合わせ、確定は必ず人が承認します。読み間違いがあっても、帳簿に入る前に止まります。

Q2. では、当面はどう運用するのですか？（付録17）

本番では、最初はAIを使いません。日付と金額は経理担当が写真を見て入れ、通帳の明細と突き合わせます。

並行してテスト環境で、個人情報を消した領収書を100枚使い、日付・金額・店名・分類ごとの正解率を測ります。合格した項目だけ、AIの候補を下書きの初期値として使い始めます。基準は、測ってから決めます。

Q3. 読み取りの精度を上げる工夫はありますか？（付録17・18）

5つあります。1つ目は「読まない」。通帳とカードはデータで取り込みます。2つ目、撮り直す。その場で撮るので色あせやしわの前に撮れ、ブレや暗さはその場で撮り直してもらいます。3つ目、絞る。AIに読ませるのは日付・合計・店名・分類の候補だけです。4つ目、確かめる。検算と、明細・現金との突き合わせです。

5つ目が、覚える。人が直した前と後、そして直した理由を会社ごとに貯めて、次の読み取りで例としてAIに渡します。モデルを再学習するのではなく、間違いを次の正解に変える仕組みです。

Q4. 周りに、OCRの精度で困っている人はいますか？

正直に言うと、まだ確かめられていません。

12月のヒアリングで、経理担当と、知り合いの家族経営の会社に聞きます。「困っていますか」ではなく、「先月、読み取りを手で直した領収書は何枚ですか」「1枚直すのに何秒かかりますか」と、実際の行動を聞きます。その数字で、OCRをどこまで入れるかを決めます。

Q5. freeeやマネーフォワードと、何が違うのですか？（4枚目・付録18）

撮影して送る機能や、銀行明細の取り込みは、会計ソフトにもあります。機能では勝負しません。

会計ソフトは「どう速く入力するか」で競っています。私は問いを変えました。「入力しなくていいデータはないか」。答えは通帳の明細です。「レシートを一番早く持っているのは誰か」。答えは現場の人です。（付録18）

だから違いは、使う人の分け方です。会計ソフトは、経理担当1人の道具です。torudakeは、現場の人は送るだけ、経理担当はつないで直すだけ、社長は確かめて締めるだけ、と家族の1人ひとりに仕事を1つずつ分けます。決算書や申告は、これまでどおり税理士や会計ソフトに任せます。torudakeは、その手前の入口です。

Q6. アプリを作る前に、どう確かめるのですか？（付録19）

アプリは作らずに、1社・2週間・0円で試します。LINEのグループに、レシートを撮って送ってもらうだけです。

測るのは、通帳の明細のうち、写真が届いた割合と、届くまでの日数です。現場の人がその場で送ってくれなければ、どんなアプリを作っても回りません。いちばん大事な仮説を、いちばん安く確かめます。

Q7. 税理士にとっては、どうなりますか？（付録19）

まだ税理士には聞いていないので、仮説です。

今は、領収書の束をまとめて受け取ることが多いと思います。torudakeなら、締めた月ごとに、証憑付きで整った資料が届きます。会計ソフトや税理士の仕事を置き換えるのではなく、その手前を整えます。確かめるために、ヒアリングで税理士にも話を聞きます。

Q8. 写真をAIに送って、情報は漏れませんか？（付録19）

正直に言うと、漏えいの心配は「なくなる」ではなく「減る」です。

写真は入力のための一時データとして、30日で自動的に消します。バックアップも30日に合わせます。正本は紙の領収書で、これまでどおり保存します。ただ、帳簿の金額や店名は残りますし、写真はAIに送ります。だから社長の同意を取り、送ったデータの扱いをAnthropicの規約で確認してから使います。

Q9. AIが間違えたまま承認してしまったら？（付録18）

承認済みの仕訳は、書き換えません。AIが「取消の仕訳」と「正しい仕訳」の案を作り、社長が承認して直します。履歴は全部残ります。

直した理由は「読み間違い」「分類違い」など選ぶだけで残り、次からAIが同じ間違いをしにくくなります。誰が間違えたかではなく、次に何を変えるかを残します。

Q10. どうやって本番で安全に動かすのですか？ AIは何を使いますか？（付録18・22）

開発・テスト・本番の3つの環境に分けます。開発は架空のデータ、テストは架空のデータと個人情報を消した領収書で確かめ、毎回バックアップを取ってから本番に出します。AIにも開発者にも、本番の削除権限は渡しません。最終判断のハンコは人が持ちます。

AIはClaudeを使います。既定はClaude Opus 5.5で、費用と精度を測ってからモデルを決めます。